

VNBOT // PRODUCT REQUIREMENTS
DOCUMENT // V1.0.0-DRAFT

VNBOT PRD

Memoria personal open source, recordatorios confiables y mini-agentes bajo el control del usuario.

Documento canónico de definición de producto. Establece qué es VNBOT, para quién se construye, qué problemas resuelve, qué funcionalidades forman parte del MVP, cuáles son sus límites explícitos y cómo se comprobará que cada versión cumple su propósito. Referencia para evitar que el producto se convierta en una colección desordenada de funciones de IA.

OPEN SOURCE

SELF-HOSTABLE

EXTENSIBLE

PRIVATE BY DESIGN

DOCUMENTO

01-PRD-VNBOT.md

VERSIÓN

1.0.0-draft

FECHA

2026-07-16

LICENCIA

MIT (prevista)

Tabla de Contenidos

1. Propósito del documento	5
2. Resumen ejecutivo	5
3. Visión del producto	6
3.1. Visión a largo plazo	6
3.2. Declaración de valor	6
3.3. Promesa principal del MVP	6
4. Problema que se desea resolver	6
4.1. Fragmentación de información	6
4.2. Recordatorios débiles	7
4.3. Asistentes sin continuidad	7
4.4. Falta de control y privacidad	7
4.5. Automatizaciones frágiles	7
5. Objetivos del producto	8
5.1. Objetivos del MVP	8
5.2. Objetivos posteriores al MVP	8
6. No objetivos del MVP	9
7. Usuarios objetivo	9
8. Casos de uso principales	10
9. Requisitos funcionales	11
9.1. Cuenta y espacios	12

9.2. Chat 12

9.3. Memoria y grafo 13

9.4. Recordatorios 13

9.5. Agentes y skills 14

9.6. MCP e integraciones 14

9.7. Distribución 14

10. Requisitos no funcionales 15

10.1. Privacidad 15

10.2. Seguridad 15

10.3. Disponibilidad 15

10.4. Rendimiento 15

10.5. Accesibilidad 16

10.6. Portabilidad 16

11. Diseño de privacidad del producto 16

11.1. Modo Local Estricto 16

11.2. Modo Servidor Privado 17

11.3. Modo LLM Externo 17

12. Experiencia visual como requisito de producto 17

12.1. Mascota principal 17

12.2. Mascotas por agente 17

12.3. Estados de la mascota 18

12.4. UI y paneles HUD 18

13. Métricas de éxito 18

13.1. Métricas de producto	18
13.2. Métricas de confiabilidad	19
13.3. Métricas de comunidad	19
14. Riesgos de producto y mitigaciones	19
15. Roadmap de producto	20
16. Criterios de lanzamiento del MVP	21
17. Backlog inicial priorizado	21
P0 — Imprescindible	21
P1 — Necesario para beta	22
P2 — Expansión	22
P3 — Comunidad y escala	22
18. Fallback heurístico sin LLM — Experiencia mínima aceptable	22
18.1. Alcance del fallback	23
18.2. Límites del fallback	23
18.3. Criterios de aceptación	23
19. Requisitos de accesibilidad	24
19.1. Estándar objetivo	24
19.2. Métricas concretas	24
19.3. Plan de auditoría de accesibilidad	24
20. Definición de "hecho" de una funcionalidad	24
21. Decisiones abiertas	25
22. Relación con otros documentos	25

23. Conclusión

26

1. Propósito del documento

Este documento define qué es VNBOT, para quién se construye, qué problemas resuelve, qué funcionalidades deben formar parte del producto, cuáles son sus límites y cómo se comprobará que cada versión cumple su propósito. Es la referencia canónica para evitar que el producto se convierta en una colección desordenada de funciones de IA.

El PRD no describe en detalle la implementación interna. Las decisiones de infraestructura, modelos, servicios, protocolos y despliegue se documentan principalmente en el TRD, el Esquema Backend y el Plan de Implementación. Este documento se centra en el "qué" y el "para quién", no en el "cómo".

El producto puede crecer de manera amplia, pero cada capacidad debe estar subordinada a cuatro objetivos fundamentales:

- **Capturar información** con el menor esfuerzo posible.
- **Convertirla** en memoria, tarea, evento o recordatorio útil.
- **Recuperarla** de forma confiable y contextual.
- **Mantener al usuario en control** de sus datos y de las acciones del asistente.

2. Resumen ejecutivo

VNBOT será un asistente personal open source, privado, autoalojable y extensible que funcionará como una capa de memoria y ejecución sobre las herramientas del usuario. El usuario podrá escribir, hablar, enviar una imagen o compartir un archivo. VNBOT interpretará la entrada y propondrá una estructura útil: memoria, recordatorio, tarea, lista, evento, relación entre entidades, borrador de acción o consulta sobre información previa.

La memoria personal se representará mediante un grafo limitado de nodos y relaciones. El usuario podrá inspeccionarlo, corregirlo, eliminarlo y exportarlo. Para ampliar la información fuera del núcleo, VNBOT podrá conectarse a herramientas externas mediante MCP, incluyendo calendarios, correo, archivos y repositorios de código como Graphify.

La identidad visual será propia y reconocible, combinando pixel art de alta calidad con un golem informático como mascota principal. Cada agente tendrá su variante coherente de la familia de golems, con estados visuales sincronizados con el funcionamiento real del sistema. Los paneles HUD cyberpunk angulares y la interfaz legible **优先**izan la accesibilidad sin depender de glassmorphism o blur.

VNBOT se distribuirá mediante múltiples canales para alcanzar distintos perfiles de usuario:

- Demo estática en GitHub Pages para evaluación sin instalación.
- Aplicación web/PWA para uso diario en navegador.

- APK Android para dispositivos móviles.
- Aplicación de escritorio para Windows, macOS y Linux.
- Imágenes Docker para servidores privados.
- CLI para usuarios avanzados y administradores.

3. Visión del producto

3.1. Visión a largo plazo

VNBOT aspira a ser una plataforma personal de memoria y agentes donde cada usuario pueda decidir qué información se almacena, qué se olvida, qué modelo de IA se utiliza, qué agentes existen, qué herramientas puede usar cada agente, qué acciones necesitan confirmación, dónde viven sus datos y si el procesamiento se realiza localmente o mediante un proveedor externo.

La visión no es crear una caja negra que actúe de manera ilimitada sin supervisión. La visión es crear una plataforma abierta y modular que permita capacidades muy amplias con permisos visibles, revocables y auditables. La autonomía avanzada se implementará después de que la memoria, los recordatorios, los jobs y la auditoría sean confiables.

3.2. Declaración de valor

VNBOT ayuda al usuario a sacar información de su cabeza, convertirla en estructuras útiles y volver a encontrarla en el momento correcto, sin obligarlo a entregar el control total de sus datos a un servicio cerrado.

3.3. Promesa principal del MVP

Escribe o dicta una instrucción en lenguaje natural; VNBOT la entiende, te muestra lo que va a hacer, la guarda correctamente y te la presenta cuando corresponda.

4. Problema que se desea resolver

4.1. Fragmentación de información

Las personas guardan información en múltiples lugares: aplicaciones de notas, mensajería, correo electrónico, calendarios, capturas de pantalla, archivos locales, historial del navegador, documentos y memoria informal. El problema no es solamente que existan muchas aplicaciones. El problema es que cada aplicación conserva una parte aislada del contexto y el usuario debe recordar dónde guardó cada cosa.

VNBOT aborda esta fragmentación ofreciendo un único punto de captura que luego distribuye la información a las estructuras apropiadas (memoria, recordatorio, tarea, lista) según la intención del usuario. La información sigue viviendo en su formato natural, pero el punto de entrada y recuperación es único y consultable.

4.2. Recordatorios débiles

Las aplicaciones tradicionales suelen requerir que el usuario complete formularios, elija una fecha, seleccione una repetición y defina una notificación. Esto funciona para tareas planificadas, pero falla cuando la idea aparece en medio de una conversación. Además, un recordatorio aislado no siempre contiene el contexto necesario: con quién está relacionado, por qué importa, qué ocurrió antes, qué debe suceder después y qué canal es adecuado.

4.3. Asistentes sin continuidad

La mayoría de los chats de IA pueden responder bien en una sesión, pero olvidan con facilidad las preferencias del usuario, las relaciones entre personas, las decisiones anteriores, las reglas de trabajo, las tareas pendientes y las correcciones hechas anteriormente. VNBOT debe tratar la memoria como una entidad visible y administrable, no como contexto invisible acumulado sin controles.

4.4. Falta de control y privacidad

Los usuarios pueden enviar información sensible a un asistente sin saber qué se almacena, qué proveedor lo procesa, cuánto tiempo se conserva, qué integraciones tienen acceso, qué datos se usan para generar embeddings y qué acciones puede ejecutar el sistema. VNBOT debe hacer visible esta información y ofrecer un modo local o autoalojable como alternativa real al cloud cerrado.

4.5. Automatizaciones frágiles

Una automatización incorrecta puede crear una fecha equivocada, enviar un correo a la persona equivocada, exponer información privada, crear recordatorios duplicados o borrar y modificar datos sin intención. Por eso, VNBOT debe separar claramente las fases del ciclo de operación para que ninguna acción se ejecute sin validación y confirmación explícita cuando tiene consecuencias.

Ciclo de operación: Interpretar → Proponer → Validar → Confirmar → Ejecutar → Auditar

5. Objetivos del producto

5.1. Objetivos del MVP

ID	Objetivo	Descripción	Criterio de éxito
0-01	Captura rápida	Permitir que un usuario registre una idea, tarea o recordatorio en lenguaje natural sin aprender comandos especiales.	Un usuario nuevo debe poder crear su primer recordatorio sin consultar documentación.
0-02	Recordatorios confiables	Crear un sistema de recordatorios que sobreviva a cierres, reinicios, fallos temporales y reintentos del worker.	No deben producirse duplicados cuando un job sea reintentado.
0-03	Memoria controlable	Permitir guardar, consultar, editar, corregir y olvidar memorias.	El usuario debe poder identificar por qué existe una memoria y eliminarla sin intervención administrativa.
0-04	Grafo comprensible	Representar relaciones personales de forma visual y limitada, sin convertir el panel en un diagrama ilegible.	Una consulta debe mostrar únicamente el subgrafo relevante, con alternativa en forma de lista.
0-05	Privacidad verificable	Ofrecer diferentes modos de procesamiento y explicar claramente qué datos salen del dispositivo.	Cada integración y proveedor debe indicar el tipo de datos que puede recibir.
0-06	Independencia de proveedor	Permitir usar un LLM local, un proveedor externo o un endpoint compatible con OpenAI.	El núcleo debe funcionar con heurísticas básicas incluso sin LLM.
0-07	Base extensible	Permitir que agentes, skills y conectores MCP se añadan sin modificar todo el núcleo.	Una integración nueva debe implementarse como adaptador o plugin sin reescribir memoria y recordatorios.
0-08	Distribución multiplataforma	Preparar el producto para web, APK, desktop, Docker y CLI.	Las interfaces deben consumir contratos API comunes y no duplicar las reglas de negocio.

5.2. Objetivos posteriores al MVP

Una vez consolidado el MVP, VNBOT incorporará gradualmente las siguientes capacidades:

- Procesamiento de voz local y remoto con transcripción editable.
- OCR e interpretación de imágenes para captura visual.
- Briefings diarios y semanales generados desde la memoria.
- Integración oficial con calendarios ICS/CalDAV.
- Lectura y borradores de correo (no envío automático en MVP).

- Telegram y WhatsApp Business API como canales de notificación.
- Agentes personalizados con mascota, skills y presupuesto propios.
- Skills comunitarias firmadas con sistema de revisión.
- Grafo temporal avanzado con línea de tiempo.
- Sincronización entre dispositivos con resolución de conflictos.
- Espacios familiares o de equipo con permisos granulares.

6. No objetivos del MVP

Los siguientes elementos quedan fuera del primer lanzamiento aunque se contemplen en la visión futura. Cada exclusión es deliberada y responde a principios de seguridad, estabilidad o madurez del producto.

#	No objetivo	Justificación
6.1	Agente autónomo general	VNBOT no podrá navegar libremente por el sistema del usuario ni ejecutar cualquier acción disponible. La autonomía se añadirá gradualmente y estará limitada por herramientas, scopes, presupuesto y confirmaciones.
6.2	Mensajería automática a terceros	El MVP no enviará mensajes a familiares, compañeros o clientes sin una configuración explícita y una política de consentimiento.
6.3	Acceso completo al correo	La primera integración de correo debe comenzar con lectura limitada o creación de borradores. El envío automático no será parte del MVP.
6.4	Operaciones financieras	No se permitirán pagos, transferencias, compras ni acciones bancarias. Esta restricción es permanente por seguridad.
6.5	Vigilancia continua del micrófono	La captura de audio será explícita y visible. No habrá escucha permanente en segundo plano en la primera versión.
6.6	Grafo ilimitado	El grafo visible tendrá límites de profundidad, cantidad de nodos y filtros. La escalabilidad de datos no implica renderizar todos los nodos simultáneamente.
6.7	Marketplace abierto de skills	Primero se necesita un sistema seguro de manifest, permisos, versionado y revisión. El marketplace queda para una etapa posterior.

7. Usuarios objetivo

VNBOT se construye para siete perfiles principales. Cada perfil tiene necesidades distintas y el producto debe servirlos sin convertirse en una herramienta solo para técnicos ni solo para casuales.

#	Perfil	Necesidades principales
7.1	Usuario personal con alta carga mental	Necesita recordar citas, pagos, compras, compromisos, ideas y tareas pequeñas. Valora una captura rápida y notificaciones confiables.
7.2	Profesional o freelancer	Gestiona proyectos, clientes, entregas, reuniones y seguimientos. Necesita relacionar personas, tareas y fechas en un mismo espacio.
7.3	Estudiante	Guarda apuntes, fechas de exámenes, temas de estudio, enlaces y recordatorios recurrentes. Beneficiado por la organización automática.
7.4	Usuario técnico	Quiere integrar VNBOT con repositorios, terminal, servidores, modelos locales y MCP. Valora la CLI, los logs estructurados y la auditabilidad.
7.5	Usuario orientado a privacidad	Prefiere ejecutar el sistema en su propio equipo o servidor, elegir el modelo y exportar todos sus datos. El modo local estricto es su flujo principal.
7.6	Administrador autoalojado	Necesita Docker, healthchecks, backups, logs, migraciones y una instalación reproducible. Documentación clara de despliegue es esencial.
7.7	Usuarios fuera del foco inicial	No se priorizan: grandes empresas con compliance corporativo, automatización bancaria, atención médica automatizada, supervisión de empleados, comunicación masiva y sistemas críticos de seguridad física.

8. Casos de uso principales

Los siguientes 10 casos de uso definen los recorridos críticos que VNBOT debe soportar desde el MVP. Cada caso tiene una entrada típica y un resultado esperado que debe cumplirse para considerar la funcionalidad terminada.

ID	Caso de uso	Entrada típica	Resultado esperado
CU-01	Crear un recordatorio	"Recuérdame pagar la electricidad mañana a las 8 de la mañana."	Detectar intención, resolver zona horaria, mostrar fecha completa, solicitar confirmación si falta información, crear recordatorio y ocurrencia, confirmar al usuario.
CU-02	Recordatorio recurrente	"Todos los lunes recuérdame revisar el presupuesto."	Crear regla de recurrencia (no lista de recordatorios duplicados). La regla debe poder pausarse, modificarse y cancelarse.
CU-03	Guardar una memoria	"Guarda que Daniel prefiere que las reuniones sean por la tarde."	Crear o actualizar nodos de persona y preferencia, relacionarlos con procedencia explícita y permitir editar el hecho.

ID	Caso de uso	Entrada típica	Resultado esperado
CU-04	Consultar contexto	"¿Qué tenía pendiente con Daniel?"	Buscar memorias, tareas y recordatorios relacionados, mostrar fuentes y distinguir hechos confirmados de inferencias.
CU-05	Captura por audio	Nota de voz del usuario.	Solicitar permisos, transcribir, mostrar texto editable y ejecutar el mismo flujo del chat. El sistema no debe actuar sobre una transcripción no revisada si la acción tiene riesgo medio o alto.
CU-06	Explorar el grafo	El usuario abre Memoria/Grafo.	Mostrar mapa limitado de nodos, filtros, búsqueda, detalle, procedencia y acción de olvidar/corregir.
CU-07	Conectar Graphify	El usuario añade un servidor MCP de Graphify.	Mostrar origen, transporte, herramientas, scopes, healthcheck, riesgos y confirmación antes de activar. Graphify no debe recibir datos personales sin permiso específico.
CU-08	Crear un agente	El usuario configura un agente de estudio.	Seleccionar mascota, instrucciones, modelo, skills, memoria permitida, herramientas y nivel de autonomía. El sistema debe mostrar un resumen de permisos antes de activar.
CU-09	Modo offline	El usuario pierde conexión.	Crear capturas y recordatorios locales, marcar sincronización pendiente y sincronizar posteriormente con idempotency keys.
CU-10	Exportar y olvidar	El usuario solicita exportar y eliminar su cuenta.	Generar backup cifrado, pedir confirmación fuerte, revocar integraciones, eliminar datos activos y presentar el resultado de la operación.

9. Requisitos funcionales

Los requisitos funcionales se organizan en siete categorías. La prioridad sigue el modelo MoSCoW: Must (imprescindible para MVP), Should (importante, puede esperar al MVP+1), Could (deseable si hay tiempo).

9.1. Cuenta y espacios

ID	Requisito	Prioridad
FR-001	Permitir modo local sin cuenta remota	Must
FR-002	Permitir cuenta en servidor privado	Must
FR-003	Separar usuarios y workspaces	Must
FR-004	Configurar zona horaria e idioma	Must
FR-005	Bloquear automáticamente la bóveda	Should
FR-006	MFA/WebAuthn	Should

9.2. Chat

ID	Requisito	Prioridad
FR-010	Chat de texto	Must
FR-011	Mostrar estado del agente	Must
FR-012	Mostrar acción propuesta	Must
FR-013	Editar propuesta antes de ejecutar	Must
FR-014	Mostrar fuentes y memorias utilizadas	Must
FR-015	Adjuntar audio	Should
FR-016	Adjuntar imagen/documento	Could
FR-017	Streaming de respuesta	Should

9.3. Memoria y grafo

ID	Requisito	Prioridad
FR-020	Crear memoria explícita	Must
FR-021	Crear nodos y relaciones	Must
FR-022	Buscar por texto	Must
FR-023	Buscar semánticamente	Should
FR-024	Ver procedencia	Must
FR-025	Editar nodo	Must
FR-026	Corregir contradicción	Should
FR-027	Olvidar nodo y relaciones	Must
FR-028	Ver subgrafo limitado	Must
FR-029	Vista alternativa en lista	Must
FR-030	Importar/exportar grafo	Must

9.4. Recordatorios

ID	Requisito	Prioridad
FR-040	Crear recordatorio puntual	Must
FR-041	Crear recurrencia	Must
FR-042	Resolver zona horaria	Must
FR-043	Detectar ambigüedad	Must
FR-044	Posponer	Must
FR-045	Completar	Must
FR-046	Cancelar	Must
FR-047	Reintentar entrega	Must
FR-048	Evitar duplicados	Must
FR-049	Historial de entregas	Should
FR-050	Ventanas de silencio	Should

9.5. Agentes y skills

ID	Requisito	Prioridad
FR-060	Crear agente	Should
FR-061	Seleccionar modelo	Should
FR-062	Asignar skills	Should
FR-063	Asignar herramientas	Should
FR-064	Limitar memoria accesible	Should
FR-065	Definir autonomía	Should
FR-066	Mostrar permisos antes de activar	Must
FR-067	Revocar herramienta	Must

9.6. MCP e integraciones

ID	Requisito	Prioridad
FR-070	Registrar servidor MCP	Should
FR-071	Realizar handshake/healthcheck	Should
FR-072	Mostrar tools y resources	Should
FR-073	Seleccionar scopes	Should
FR-074	Confirmar operaciones de riesgo	Must
FR-075	Desconectar y revocar credenciales	Must
FR-076	Registrar ejecución	Must

9.7. Distribución

ID	Requisito	Prioridad
FR-080	Demo mock en GitHub Pages	Must
FR-081	Docker Compose local	Must
FR-082	Build APK	Should
FR-083	Build desktop	Should
FR-084	CLI de diagnóstico	Should
FR-085	Exportación portable	Must

10. Requisitos no funcionales

10.1. Privacidad

- El modo local no debe enviar contenido fuera del dispositivo salvo activación explícita.
- Cada proveedor externo debe estar identificado con nombre, modelo y política de datos.
- No se debe presentar como zero-knowledge un flujo donde el proveedor pueda leer plaintext.
- El usuario debe poder eliminar memorias, archivos, embeddings y logs asociados.

10.2. Seguridad

- Contraseñas con Argon2id y parámetros robustos.
- Cookies seguras (HttpOnly, Secure, SameSite) y sesiones revocables.
- Validación de todos los payloads entrantes con schema estricto.
- CSP y sanitización de contenido generado por LLM para prevenir XSS.
- Protección SSRF para webhooks y conexiones MCP salientes.
- Rate limiting distribuido en servidor (por IP, por usuario, por endpoint).
- Tokens de integración separados por scope y rotables.
- Auditoría de acciones con timestamp, actor y resultado.
- Escaneo de dependencias y secretos en CI.

10.3. Disponibilidad

Un recordatorio confirmado debe sobrevivir a los siguientes eventos sin perderse ni duplicarse:

- Reinicio de API.
- Reinicio de worker.
- Reintento del job después de un fallo temporal.
- Pérdida temporal de red.
- Actualización de contenedor.

10.4. Rendimiento

Objetivos iniciales de rendimiento para una instalación personal razonable:

Métrica	Objetivo
Primera respuesta de interfaz local	< 200 ms (con datos en cache)

Métrica	Objetivo
Búsqueda textual local	P95 < 300 ms para 10.000 memorias
Búsqueda híbrida (textual + semántica)	P95 < 1 segundo
Creación de job de audio	< 500 ms (transcripción asíncrona)
Grafo inicial	< 1 segundo para hasta 100 nodos visibles

10.5. Accesibilidad

- Contraste mínimo AA (4.5:1 texto normal, 3:1 texto grande).
- Navegación completa por teclado (Tab, Enter, Esc, atajos).
- Etiquetas ARIA para todos los controles interactivos.
- Estados no comunicados solo por color (icono + texto).
- Respeto a prefers-reduced-motion.
- Alternativa de lista para el grafo visual.
- Texto legible en pantallas pequeñas (mínimo 14px móvil).

10.6. Portabilidad

- Chrome, Firefox y Edge modernos (últimas 2 versiones mayores).
- Android soportado por el APK definido para cada release.
- Windows, Linux y macOS según matriz de builds publicada.
- Docker en Linux x64 y ARM64 cuando sea posible.
- SQLite para modo local y desktop.
- PostgreSQL para servidor.

11. Diseño de privacidad del producto

VNBOT debe presentar tres modos comprensibles de operación. La elección del modo es explícita por parte del usuario durante el onboarding y se puede cambiar posteriormente en la configuración. Cada modo tiene implicaciones claras que deben comunicarse sin ambigüedad.

11.1. Modo Local Estricto

Todas las operaciones se realizan en el dispositivo del usuario. Sin dependencias externas.

- Memoria almacenada en dispositivo (SQLite o IndexedDB).
- Embeddings generados localmente.

- Audio procesado localmente.
- LLM local vía Ollama, llama.cpp o modelo integrado.
- Sin sincronización remota por defecto.
- Exportación manual para transferir datos.

11.2. Modo Servidor Privado

Instalación propia en servidor elegido por el usuario. Sync entre dispositivos.

- API y base de datos en servidor elegido por el usuario.
- El administrador del servidor puede tener acceso técnico al proceso.
- Debe explicarse la diferencia entre servidor propio y zero-knowledge.
- Integraciones y backups controlados por el propietario del servidor.

11.3. Modo LLM Externo

Uso de proveedores LLM externos (OpenAI, Anthropic, etc.) con contexto mínimo.

- El proveedor LLM recibe el contexto mínimo necesario para la operación.
- El usuario selecciona proveedor y modelo explícitamente.
- Se muestran proveedor, modelo y política de datos del proveedor.
- Se pueden bloquear memorias sensibles para proveedores externos.

12. Experiencia visual como requisito de producto

La estética no será una capa decorativa añadida al final. Será parte de la identidad funcional de VNBOT. La mascota, los estados visuales, los paneles HUD y la tipografía comunican información operativa real, no solo decoración.

12.1. Mascota principal

El golem informático será la representación de VNBOT. Debe ser reconocible a 32x32, 64x64 y 128x128 píxeles. La mascota es el feedback visual principal del estado del sistema y debe cambiar su pose, visor y animación según las operaciones en curso.

12.2. Mascotas por agente

Cada agente tendrá una variante coherente de la familia de golems. La variación se expresa mediante:

- Visor (color y patrón).

- Accesorio (antena, escudo, lentes, drones).
- Paleta (cyan, amber, magenta, violet, green, blue, white).
- Silueta secundaria (proporciones del cuerpo).
- Animación (idle, hover, processing).

12.3. Estados de la mascota

La mascota indicará el estado real del sistema mediante 10 estados visuales:

- Escuchando (listening) — captura de audio/entrada activa.
- Pensando (thinking) — clasificación o recuperación en curso.
- Procesando (processing) — job activo con anillo multicolor.
- Esperando confirmación (waiting_confirmation) — propuesta pendiente.
- Ejecutando (executing) — acción en curso.
- Completado (success) — operación finalizada con éxito.
- Advertencia (warning) — resultado parcial o configuración requerida.
- Error (error) — job fallido, postura de alerta.
- Desconectado (offline) — sin conexión, visor apagado.
- Durmiendo (sleeping) — baja actividad, pose compacta.

Toda animación debe acompañarse de texto accesible o estado ARIA. La mascota nunca es el único canal de información.

12.4. UI y paneles HUD

Los paneles HUD, las retículas y los marcos pixel art no deben impedir la lectura. Los mensajes, formularios, permisos y logs deben priorizar claridad sobre decoración. La interfaz prioriza legibilidad, accesibilidad y rendimiento en dispositivos móviles.

13. Métricas de éxito

13.1. Métricas de producto

- Tiempo hasta el primer recordatorio (TTFR).
- Porcentaje de usuarios que completan onboarding.
- Porcentaje de recordatorios creados correctamente en el primer intento.
- Tasa de memorias encontradas en consultas de prueba.
- Número de memorias corregidas o eliminadas por el usuario.
- Uso de exportación (frecuencia y alcance).

- Activación de modo local vs servidor.
- Número de agentes configurados por usuario.

13.2. Métricas de confiabilidad

- Entregas correctas (porcentaje de ocurrencias entregadas a tiempo).
- Duplicados por cada 10.000 ocurrencias (objetivo: 0).
- Jobs perdidos (objetivo: 0).
- Jobs reintentados con éxito (tasa de recuperación).
- Tiempo medio de recuperación (MTTR) tras fallo.
- Estado de backups (última verificación exitosa).

13.3. Métricas de comunidad

- Contribuidores activos por mes.
- Pull requests aceptados por mes.
- Issues resueltos por mes.
- Plugins/skills revisados y publicados.
- Instalaciones Docker activas (opt-in telemetry).
- Descargas de Releases por plataforma.
- Documentación consultada (páginas más visitadas).

Ninguna métrica de producto debe requerir enviar el contenido privado del usuario a un servicio de analítica. Las métricas agregadas son opt-in y nunca incluyen contenido de memorias.

14. Riesgos de producto y mitigaciones

Cada riesgo identificado tiene una mitigación concreta. Los riesgos de impacto alto requieren mitigación obligatoria antes del lanzamiento del MVP.

Riesgo	Impacto	Mitigación
Fecha mal interpretada	Alto	Fecha completa visible y confirmación explícita antes de crear recordatorio.
Memoria inventada por alucinación del LLM	Alto	Procedencia obligatoria, confianza calculada y respuesta basada en fuentes verificadas.
Exceso de notificaciones	Medio	Ventanas de silencio, agrupación y aprendizaje controlado de preferencias.

Riesgo	Impacto	Mitigación
Costes LLM descontrolados	Medio	Router con fallback a modelos locales, límites por usuario y presupuestos por agente.
MCP malicioso	Alto	Allowlist de servidores, scopes granulares, sandbox y revocación inmediata.
Complejidad excesiva	Alto	MVP pequeño y arquitectura por plugins. Cada nueva capacidad pasa por revisión.
Assets incompatibles con licencias	Medio	Inventario y auditoría de licencias antes de cada release.
Pérdida de datos	Alto	Backups automáticos, exportación manual y pruebas de restore verificadas.
Falsa promesa de privacidad	Alto	Documentar claramente cada modo sin usar zero-knowledge como marketing genérico.
Dependencia de una plataforma	Medio	Adaptadores y canales oficiales. Núcleo no acoplado a un solo proveedor.

15. Roadmap de producto

El roadmap se organiza en 8 versiones desde la demo hasta la plataforma estable. Cada versión tiene criterios de salida claros y no se avanza sin validar la anterior.

Versión	Nombre	Entregables principales
0.1	Demo	Landing en GitHub Pages, chat simulado, grafo con datos ficticios, mascota y estados visuales, documentación inicial.
0.2	Local	PWA, IndexedDB, bóveda local, recordatorios locales, heurística sin LLM, exportación.
0.3	Backend privado	FastAPI, SQLite/PostgreSQL, worker, scheduler, Docker, healthchecks, LLM Router.
0.4	Plataformas	APK, desktop, CLI, notificaciones nativas, GitHub Releases.
0.5	Memoria	Grafo real, búsqueda híbrida, procedencia, correcciones, expiración.
0.6	MCP	MCP interno, gateway externo, permisos, Graphify opcional, calendario opcional.
0.7	Agentes	Skills, agentes especializados, mascotas por agente, presupuestos, auditoría.
1.0	Plataforma estable	Instalación documentada, backups verificados, releases firmados, plugin SDK, suite de pruebas, guía de seguridad.

16. Criterios de lanzamiento del MVP

VNBOT podrá considerarse listo para una primera versión pública cuando se cumplan los siguientes 12 criterios:

01. Un usuario pueda instalarlo sin editar código fuente.
02. El flujo de captura funcione sin LLM mediante heurística básica.
03. Los recordatorios sean persistentes e idempotentes.
04. Exista una vista clara de tareas y recordatorios.
05. La memoria tenga edición, eliminación y procedencia.
06. El grafo tenga una alternativa textual.
07. El modo local no dependa de servicios cloud.
08. El sistema tenga healthchecks y diagnóstico CLI.
09. Exista exportación y restauración probada.
10. Los errores no expongan secretos ni contenido privado en logs.
11. El repositorio tenga CI, licencia, seguridad y guía de contribución.
12. La demo de GitHub Pages no solicite datos reales.

17. Backlog inicial priorizado

P0 — Imprescindible

- Crear monorepo.
- Implementar modelo de dominio.
- Crear almacenamiento local SQLite/IndexedDB.
- Crear chat base.
- Crear parser heurístico de recordatorios.
- Crear scheduler y ocurrencias.
- Crear notificación local.
- Crear CRUD de memoria.
- Crear exportación.
- Crear healthchecks.
- Crear demo mock para GitHub Pages.

P1 — Necesario para beta

- FastAPI.
- PostgreSQL.
- Redis y workers.
- LLM Router.
- Búsqueda híbrida.
- Grafo visual.
- Docker Compose.
- Audio.
- APK.
- Desktop.

P2 — Expansión

- MCP Gateway.
- Graphify adapter.
- Calendario.
- Skills.
- Agentes personalizados.
- Briefings.
- Telegram.
- Gmail como borrador.

P3 — Comunidad y escala

- Plugin SDK.
- Marketplace de skills.
- Sincronización multi-dispositivo.
- Espacios compartidos.
- Grafo temporal avanzado.
- Métricas agregadas opt-in.

18. Fallback heurístico sin LLM — Experiencia mínima aceptable

VNBOT debe funcionar sin un proveedor LLM conectado. El fallback heurístico define el piso de experiencia cuando no hay IA disponible. Este piso no es un modo degradado: es una experiencia completa y útil para las operaciones más comunes.

18.1. Alcance del fallback

El sistema heurístico debe cubrir al menos las siguientes intenciones sin LLM:

Intención	Método	Ejemplo
Crear recordatorio	Regex + parsing de fecha/hora natural	"recuérdame llamar a Ana mañana a las 5" → recordatorio puntual
Crear memoria	Captura directa sin estructura	"mi código postal es 1040" → memoria tipo fact
Consultar memorias	Búsqueda textual exacta y fuzzy	"buscar Daniel" → coincidencias por texto
Listar recordatorios	Query directa a storage	"¿qué tengo hoy?" → recordatorios del día
Completar recordatorio	Match por ID o texto parcial	"ya llamé a Ana" → marcar completado
Crear lista	Comando estructurado	"lista de compras: leche, pan" → lista con items

18.2. Límites del fallback

Lo que el fallback NO debe intentar:

- Inferir relaciones entre memorias (requiere embeddings o reglas complejas).
- Interpretar imágenes o audio (requiere modelos específicos).
- Generar resúmenes o briefings.
- Operar herramientas MCP.
- Manejar contexto conversacional multi-turno complejo.

Cuando el fallback no pueda interpretar una intención, debe responder con un mensaje claro: "No pude interpretar eso sin un modelo de IA conectado. [Instrucciones para configurar un proveedor]."

18.3. Criterios de aceptación

- Un usuario puede crear, consultar y completar recordatorios sin LLM.
- Un usuario puede capturar y buscar memorias sin LLM.
- La experiencia sin LLM está documentada en el onboarding.
- El fallback tiene tests unitarios con al menos 20 casos de entrada documentados.
- El mensaje de "no disponible sin LLM" incluye enlace a configuración.

19. Requisitos de accesibilidad

19.1. Estándar objetivo

VNBOT debe cumplir WCAG 2.2 nivel AA como mínimo en todas sus interfaces. El nivel AAA es un objetivo aspiracional para componentes críticos (chat, recordatorios, confirmaciones de acciones).

19.2. Métricas concretas

Métrica	Requisito	Verificación
Contraste de texto	Mínimo 4.5:1 (AA), 3:1 para texto grande	Lighthouse + axe-core en CI
Contraste de paneles HUD	Mínimo 3:1 entre borde de panel y fondo adyacente	Manual + herramienta
Tamaño mínimo de texto	14px en móvil, 12px solo en labels no esenciales	Design tokens
Touch targets	Mínimo 44×44px en móvil (WCAG 2.5.8)	Layout tests
Navegación por teclado	Todas las funciones accesibles sin ratón	Testing manual e2e
Screen reader	Labels ARIA en todos los componentes interactivos	axe-core + manual
Reduced motion	Respetar prefers-reduced-motion: reduce	CSS + tests visuales
Focus visible	Indicador de foco visible en todos los elementos interactivos	Automated + manual

19.3. Plan de auditoría de accesibilidad

- Auditoría inicial antes de v0.2.
- Auditoría por cada release minor (0.x).
- axe-core automatizado en CI para cada PR.
- Auditoría manual con screen reader (NVDA/VoiceOver) antes de cada release major.
- Test con usuarios con discapacidades antes de v1.0.

20. Definición de "hecho" de una funcionalidad

Una funcionalidad se considera terminada cuando cumple los siguientes 12 criterios. Ninguno es opcional:

01. Tiene requisito identificado (FR-xxx).

02. Tiene diseño UX aprobado.
03. Tiene contrato de datos/API definido.
04. Tiene estados de error definidos.
05. Tiene política de permisos definida.
06. Tiene pruebas unitarias con cobertura mínima.
07. Tiene prueba de integración.
08. Tiene documentación de usuario y técnica.
09. Tiene comportamiento offline o degradado definido.
10. No filtra secretos en logs.
11. Funciona en el entorno objetivo.
12. Tiene criterio de rollback o migración si modifica datos.

21. Decisiones abiertas

Estas decisiones deberán cerrarse en documentos técnicos posteriores (TRD, Esquema Backend, Plan de Implementación):

01. SQLite local con SQLAlchemy o una capa Rust/SQLite para desktop.
02. Capacitor frente a React Native si aumentan las funciones nativas.
03. Celery/Dramatiq frente a un worker Rust.
04. pgvector frente a un índice vectorial local dedicado.
05. WebAuthn en el MVP o en una release posterior.
06. Formato final de skills: Markdown + YAML o JSON Schema + Markdown.
07. Transporte MCP principal: stdio local y Streamable HTTP remoto.
08. Sistema de actualización automática de desktop.
09. Política final de retención de audios.
10. Compatibilidad exacta de licencias para assets generados y repositorios de referencia.

22. Relación con otros documentos

El PRD es parte de un conjunto documental coherente. Cada documento cubre una dimensión específica del proyecto:

Documento	Cobertura
Documento Maestro (00)	Visión canónica y decisiones generales del proyecto.
PRD (01)	Este documento. Definición de producto, requisitos y criterios.
TRD (02)	Decisiones técnicas y requisitos de infraestructura.
Esquema Backend (03)	Servicios, API, jobs y eventos del backend.
Plan de Implementación (04)	Fases y ejecución detallada del desarrollo.
Diseño UI/UX (05)	Interfaz, mascotas y sistema visual completo.
App Flow (06)	Recorridos, estados y transiciones de usuario.
Modelo de Datos (07)	Entidades, relaciones y persistencia.
Seguridad y Privacidad (08)	Amenazas, controles y privacidad por diseño.
MCP y Skills (09)	Herramientas, agentes y permisos.
Roadmap (10)	Releases, prioridades y dependencias temporales.

23. Conclusión

VNBOT debe empezar como un sistema pequeño y fiable, no como un agente omnipotente. El núcleo del producto es la relación entre captura, memoria y recordatorio. Los LLM, MCP, agentes, canales y assets visuales deben ampliar ese núcleo sin ocultar su funcionamiento ni debilitar la privacidad.

La primera pregunta para cada nueva funcionalidad debe ser:

¿Ayuda al usuario a recordar, encontrar o ejecutar algo de manera más confiable y controlable?

Si la respuesta es afirmativa, la funcionalidad puede incorporarse mediante un módulo, skill, integración o agente con permisos claros. Si la respuesta es negativa, debe permanecer fuera del núcleo aunque sea técnicamente atractiva. La disciplina en esta pregunta es lo que separa a VNBOT de una colección desordenada de funciones de IA.